XPP Studio

An introduction and tutorial

Peter Ashwin and Bard Ermentrout
with Claude Opus 5.5 (Anthropic)

October 2026

Abstract

XPP Studio is a browser-based environment for exploring dynamical systems, modelled on Bard Ermentrout's XPPAUT. It reads XPPAUT's `.ode` files and provides phase planes, three-dimensional views, time courses, numerical continuation of equilibria and periodic orbits, one- and two-parameter bifurcation diagrams, maps, stochastic and Markov models, stiff solvers, delay equations, events (global flags), arrays, Volterra integral equations and boundary value problems. Everything runs locally in a web page: there is nothing to install and no data leaves the browser. This document introduces the interface, describes the model language, and works through a sequence of short tutorials using the examples that come with the program.

Contents

1 Introduction **3**

2 A tour of the interface **3**

3 The model language **5**

4 Tutorial 1: phase-plane analysis **7**

5 Tutorial 2: one-parameter bifurcation diagrams **8**

6 Tutorial 3: two-parameter continuation **10**

7 Tutorial 4: chaos in three dimensions **13**

8 Tutorial 5: maps **13**

9 Tutorial 6: noise and random switching **14**

10 Tutorial 7: stiff systems **15**

11 Tutorial 8: arrays and Volterra equations **16**

12 Tutorial 9: delay equations **19**

13 Tutorial 10: events and global flags **22**

14 Tutorial 11: boundary value problems **23**

15 Numerical settings **27**

16 Saving and exporting results **29**

17 Differences from XPPAUT **30**

18 Credits and licence **30**

1 Introduction

For more than thirty years XPPAUT [1] has been one of the standard tools for simulating and analysing differential equations in mathematical biology, neuroscience and applied dynamical systems. Its strengths are a compact, readable model language, fast interactive phase-plane analysis, and a tight connection to the continuation package AUTO [2]. XPP Studio keeps that model language and that way of working, and puts them in a modern interface:

- **Same files.** Most XPPAUT `.ode` files load unchanged. Supported features include parameters, functions, fixed quantities, auxiliary variables, maps, Wiener noise, Markov variables, tables, periodic (folded) variables, arrays, Volterra integrals, delays, global flags and boundary conditions (Section 3).
- **Live analysis.** Dragging a parameter slider immediately recomputes trajectories, nullclines and equilibria.
- **Built-in continuation.** Equilibria, periodic orbits and their bifurcations can be continued in one or two parameters without leaving the program. Detected points include folds, Hopf, period-doubling, torus, fold-of-cycles and codimension-two points. Solutions of boundary value problems are solved by shooting and can be followed in a parameter, round folds (Section 14).
- **Publication output.** Every view can be exported as a vector SVG figure. Continuation data can be exported as CSV, and trajectories copied as CSV.

The program is a single web page. To use it, open

`https://xpp-studio.github.io/xpp-studio/`

in any recent browser. The source code, the example models and this tutorial are in the repository `https://github.com/xpp-studio/xpp-studio`. Its file `index.html` is the whole program, and it also runs when opened directly from disk.

How this document is organised. Section 2 is a tour of the interface and Section 3 summarises the model language. Sections 4–14 are hands-on tutorials, each built on an example from the **Model** menu. Section 15 covers numerical settings and Section 16 exporting results. The document ends with the differences from XPPAUT and with credits.

2 A tour of the interface

The window (Figure 1) has three columns.

Left: the model. Four tabs:

Equations The `.ode` text, which you can edit freely. **Apply** (or `Ctrl`+`Enter`) reloads the model, and messages underneath report what was loaded and any errors, with line numbers. **Open .ode** loads a file from disk, **Copy text** copies the current text, **Write state** writes the current parameters, initial conditions and settings into it (Section 16), and **Table files** supplies data files used by `table` declarations.

Parameters One row per parameter, with a value box and a slider whose limits you can edit. Drag a slider to see everything update live, and use **Reset** to restore the values in the file. The up and down arrows on a value box, and the arrow keys on a slider, change the value by 1% of the slider's range (10% with `Shift`).

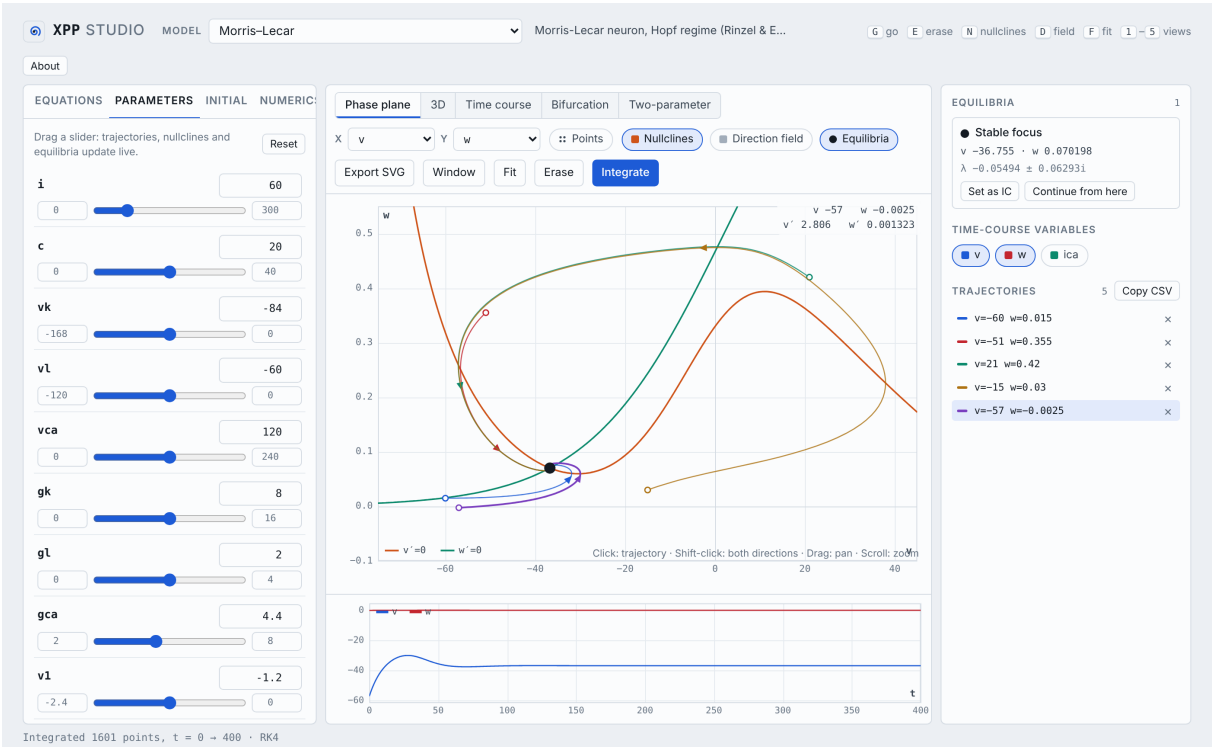


Figure 1: The XPP Studio window with the Morris–Lecar example loaded. Left: tabs for equations, parameters, initial conditions and numerics. Centre: the plot, here the (v, w) phase plane with nullclines, five trajectories and a stable focus, plus a time-course strip underneath. Right: the inspector, listing equilibria, the variables shown in the time course, and the trajectories.

Initial Initial values of the state variables (and Markov states). **Integrate** runs from here, and **Last point → IC** copies the end of the current trajectory into the initial conditions. An initial value named in a `#@irange` line also has a slider, and a model with boundary conditions lists them here with their current residuals (Section 14).

Numerics Integration method, total time, transient, step size, tolerance and bound, plus the continuation settings (Section 15).

Centre: the plot. Five views, chosen with the buttons above the plot or with the keys 1–5:

Phase plane Any two variables (state, Markov, Volterra or auxiliary) against each other, with optional nullclines, direction field and equilibria. For a one-dimensional map this view shows a cobweb diagram. **Equal axes** gives both axes the same scale. Below the plane a strip shows the time series of the current trajectory.

3D Three variables (or time) in a rotatable box: drag to rotate, scroll to zoom, **Spin** to animate.

Time course Chosen variables against t (or the iteration number n for maps). The **Spectrum** button replaces the time series by its power spectrum (Section 9), and, for a model with a row of indexed variables, **Space-time** shows the cells as an image (Section 11).

Bifurcation Branches of equilibria and periodic orbits, and parameter sweeps, against one parameter. The vertical axis is a variable, or the period or frequency of the periodic orbits. For a boundary value problem it shows the maximum and minimum of a variable, or its L^2 norm, along a branch of solutions (Section 14).

Two-parameter Curves of folds, Hopf points, period doublings and bifurcations of cycles in a parameter plane.

In every view you drag to pan and scroll to zoom. **Fit** rescales to the data, and **Window** sets the axis limits by hand (they then stay fixed until you press **Fit**). **Points** draws trajectories as dots instead of lines, which is the default for maps. **Export SVG** saves the current view as a vector figure. A model with boundary conditions also has a **Solve BVP** switch among the buttons above the plot (Section 14). **Export CSV** saves the data behind the time course, bifurcation diagram or two-parameter diagram, and **Export .dat** saves continuation data as plain columns of numbers (Section 16).

Right: the inspector. The inspector's contents depend on the view. In the phase plane it lists the equilibria (type, coordinates and eigenvalues, with buttons to make one the initial condition, start continuation from it, or draw its separatrices), the variables in the time course, and the trajectories. In the Bifurcation view it holds the continuation controls and the table of special points. In the Two-parameter view it holds the two-parameter controls.

Key	Action
G	integrate from the current initial condition ("go")
E	erase all trajectories
N / D	toggle nullclines / direction field
F	fit the axes to the data
1–5	phase plane, 3D, time course, bifurcation, two-parameter
Esc	during continuation: skip to the other direction ("soft interrupt")
Shift+Esc	stop the continuation altogether
click / shift-click	start a trajectory (shift: forwards and backwards in time)

Table 1: Keyboard shortcuts and mouse actions.

3 The model language

A model is a plain-text `.ode` file in XPPAUT's format. Lines starting with `#` are comments, and the first comment line becomes the model's title. Names are case-insensitive. The Morris–Lecar example [9, 10] illustrates most of the basic syntax:

```
# Morris-Lecar neuron, Hopf regime (Rinzel & Ermentrout)
dv/dt=(i+gl*(vl-v)+gk*w*(vk-v)+gca*minf(v)*(vca-v))/c
dw/dt=phi*(winf(v)-w)/tauw(v)
minf(v)=.5*(1+tanh((v-v1)/v2))
winf(v)=.5*(1+tanh((v-v3)/v4))
tauw(v)=1/cosh((v-v3)/(2*v4))
aux ica=gca*minf(v)*(v-vca)
param i=60,c=20
param vk=-84,vl=-60,vca=120
param gk=8,gl=2,gca=4.4
param v1=-1.2,v2=18,v3=2,v4=30,phi=.04
init v=-60,w=0.015
#@range i 0 300
@ total=400,dt=0.25,meth=rk4,xp=v,yp=w
@ xlo=-75,xhi=45,ylo=-0.1,yhi=0.55
done
```

Table 2 summarises the language. The `#@range` line is an XPP Studio extension that sets a slider's range. Because it starts with `#`, XPPAUT reads it as a comment, so files stay portable.

Syntax	Meaning
`x'=...  or  dx/dt=...`	differential equation
`x(t+1)=...`	map (or use `x'=...` with `@ meth=discrete`)
`x(0)=1  or  init x=1,y=0`	initial values
`par a=1, b=2`	parameters (also `param`, `p`)
`number c=3`	named constant
`!d=a*b`	derived parameter, recomputed when parameters change
`f(x,y)=...`	user function
`q=...`	fixed quantity, evaluated before the equations
`aux s=...`	auxiliary output, plotted like a variable
`wiener w`	white noise for an SDE, e.g. `x'=-x+s*w`
`markov z 2  {0} {kon}  {koff} {0}`	two-state Markov variable; entry (i, j) is the rate from i to j
`table h % 101 0 6.283 sin(t)`	table sampled from an expression
`table h data.tab`	table read from a file (add it with **Table files**)
`x[1..20]'=-x[j]+x[j-1]`	array: the line is repeated for $j = 1, \ldots, 20$
`%[1..20] ... %`	repeat a block of lines
`int{K(t,t',x)}`	Volterra integral $\int_0^t K(t, t', x(t'))\, dt'$
`int{K#x}`	convolution $\int_0^t K(t - t')\, x(t')\, dt'$
`int[mu]{...}`	weakly singular kernel, extra factor $(t - t')^{-\mu}$
`u(t)=...int{...}`	integral equation (also `volterra u=...`)
`delay(x,tau)`	delayed value $x(t - \tau)$; τ may depend on the state
`x(0)=0.5+0.2*cos(t)`	history of a delay equation for $t < 0$ (its value at $t = 0$ is the initial condition)
`global 1 x-1 {x=0;y=y+1}`	global flag: when `x-1` crosses zero upwards (1), downwards (-1) or either way (0), make the assignments; `arret=1` stops the run
`b u-1  b u'-2`	boundary conditions (also `bndry`): u is its value at the left end, u' at the right end, so these say $u(0) = 1$ and $u(T) = 2$ (Section 14)
`#@range p lo hi`	slider range for parameter `p`
`#@irange u lo hi`	slider range for the initial value of `u`
`@ total, t0, trans, dt, meth, toler, bound`	integration: duration, start time, transient, step, method, tolerance, bound
`@ xp, yp, zp, axes=3, xlo, xhi, ylo, yhi`	initial plot variables and window
`@ fold=th, tor_per=6.2832`	periodic variable `th` with the given period
`@ seed=7`	fix the random seed
`@ vmaxpts=4000`	memory length for Volterra integrals
`@ ds, dsmin, dsmax, nmax, ntst, parmin, parmax`	continuation numerics, as in XPPAUT's AUTO window
Functions: `sin cos tan exp ln log sqrt abs heav sign max min mod atan2 sinh cosh tanh`, `if(c)then(a)else(b)`, ...	

Table 2: The `.ode` language understood by XPP Studio.

Methods. `meth=` accepts `rk4` (fixed-step Runge–Kutta), `euler`, `dp45` (adaptive Dormand–Prince 4(5)), and the stiff names `stiff`, `gear`, `cvode`, `rosenbrock` and `backeul`. All of the stiff names select an adaptive Rosenbrock 2(3) solver [7]. `discrete` iterates a map. Models with Wiener variables always use Euler–Maruyama, and models with Volterra integrals use a product-trapezoid scheme.

4 Tutorial 1: phase-plane analysis

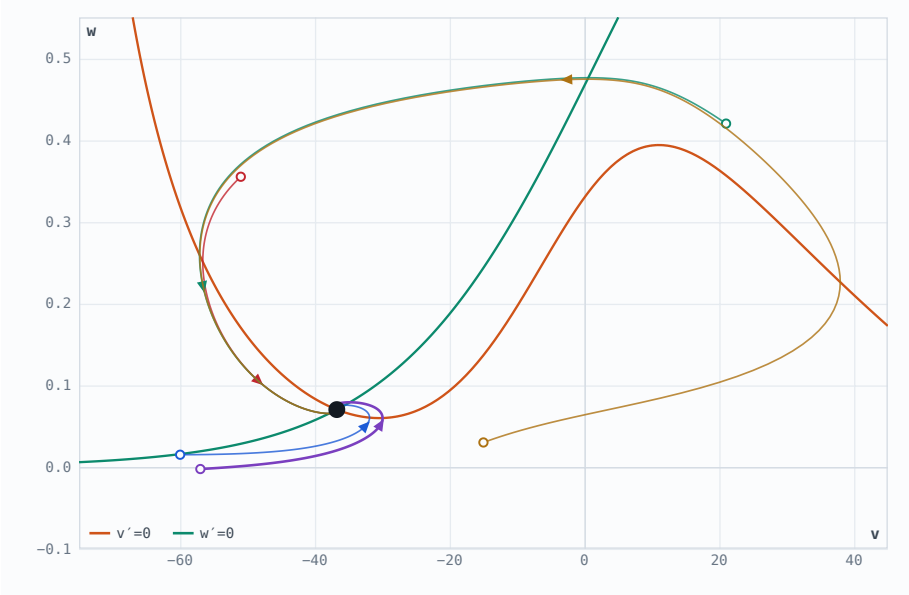


Figure 2: Morris–Lecar phase plane at $i = 60$: the v-nullcline (orange), the w-nullcline (green), a stable focus (filled circle) and several trajectories. Open circles mark starting points.

Choose **Morris–Lecar** from the **Model** menu. The phase plane shows w against the membrane potential v, together with the two nullclines.

Try it

1. Click anywhere in the plane to start a trajectory there. Shift-click integrates both forwards and backwards. Each new trajectory gets its own colour and appears in the **Trajectories** list, where you can remove it with ×.
2. Press D for the direction field and N to hide or show the nullclines. Hovering over the plot shows the coordinates and the derivatives v', w' at the pointer.
3. The inspector lists one equilibrium, a *stable focus*, with its eigenvalues $\lambda \approx -0.055 \pm 0.063i$. **Set as IC** moves the initial condition onto it.
4. Open the **Parameters** tab and slowly drag the slider for the applied current i. Between about $i = 94$ and $i = 212$ the focus becomes unstable and trajectories settle onto a large limit cycle: repetitive firing.
5. Press 3 for the time course. The chips in the inspector choose which variables are plotted, auxiliary ones included (`ica` here).

Equilibria are found by Newton's method from a grid of starting points covering the visible window, so panning or zooming out searches a wider region. For a saddle, the **Separatrices** button integrates its stable and unstable manifolds. Once drawn, they follow the saddle: when

you change a parameter, the saddle is found again and its separatrices are recomputed from its new position, so you can watch, for example, a separatrix approach a homoclinic connection as you drag a slider.

5 Tutorial 2: one-parameter bifurcation diagrams

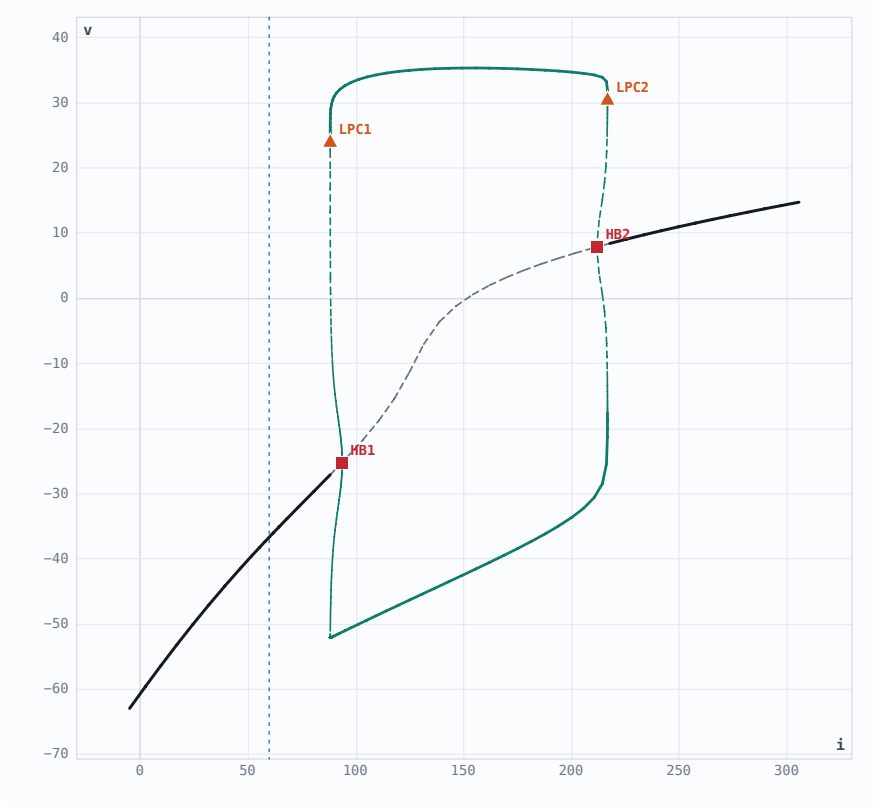


Figure 3: Bifurcation diagram of Morris–Lecar in the current i. Black: equilibria (solid stable, dashed unstable). Green: maximum and minimum of v along the periodic orbits born at the two Hopf points HB1 and HB2. Both Hopf bifurcations are subcritical. The orbit family turns at two folds of cycles, LPC1 and LPC2, giving bistability between rest and firing.

Press 4 for the Bifurcation view. The inspector shows the continuation controls: the **Parameter** to vary, the variable to **Plot**, the parameter **Range**, the initial step **Step ds**, and the equilibrium to **Start** from.

Try it

1. Press **Continue equilibria**. Pseudo-arclength continuation [6] traces the branch in both directions across the range. Stability is read off the eigenvalues, and two Hopf points appear: HB1 at $i \approx 93.68$ (period ≈ 79.2) and HB2 at $i \approx 212.06$. Each Hopf point is tagged *sub* or *super* in the **Special points** table (Figure 6). The tag comes from the sign of the first Lyapunov coefficient ℓ_1, and hovering over it shows the value. Both points here are subcritical ($\ell_1 > 0$), so the orbits born there are unstable.
2. In the **Special points** table, press **Cycles** on the row for HB1. The periodic orbits born there are computed by multiple shooting with an integral phase condition, as in AUTO, and their stability comes from Floquet multipliers. While the computation runs, Esc (or **Skip direction**) abandons the current direction and moves on, and Shift+Esc stops altogether.

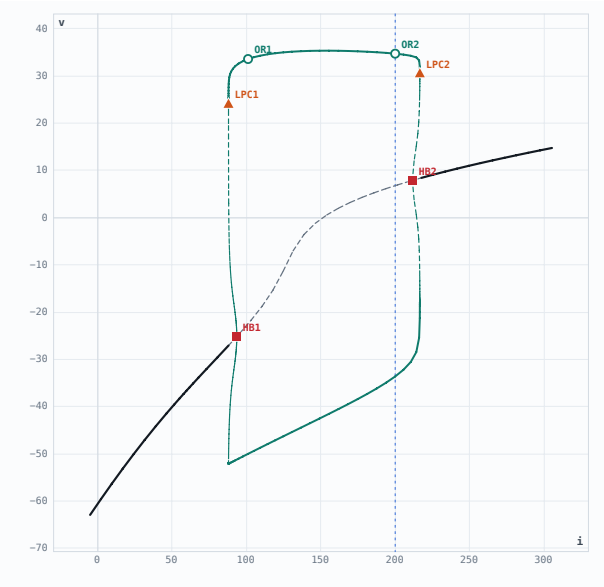
3. The family of orbits connects HB1 to HB2 and turns at two folds of cycles: LPC1 at $i \approx 88.29$ and LPC2 at $i \approx 216.90$ (Figure 3). Between LPC1 and HB1 a stable orbit and the stable rest state coexist.

4. Click any point of a branch to jump there. This sets the parameter and loads that equilibrium, or exactly one period of that orbit, as the current state. Clicking a row of the special-points table does the same.

5. **Limit cycle** starts orbit continuation from the current trajectory instead. Integrate onto a stable cycle first (e.g. at $i = 150$), then press it.

6. Choose **period** (or **frequency**) under **Plot** to draw the period of the orbits against the parameter (Figure 5). The period grows from about 79 at HB1 to 135 at LPC1 and is about 42 at HB2. Sweeps show their periods too, where the oscillation is regular enough for one to be estimated.

7. Click a point on the orbit branch. As well as loading that orbit, this adds it to the **Special points** table as OR1, marked on the diagram with an open circle (Figure 4). Click another point to add OR2, and so on. Each picked orbit has a **Const. T** button, which follows it in two parameters at fixed period (Tutorial 3), and a **Remove** button.



SPECIAL POINTS 6

PT	I	V	PERIOD	
HB1 sub	93.682	−25.321	79.25	Follow / Cycles
HB2 sub	212.06	7.8003	42.33	Follow / Cycles
LPC1	88.293	23.925	135.4	Follow
LPC2	216.9	30.397	77.93	Follow
OR1	101.44	33.496	83.99	Const. T / Remove
OR2	200.31	34.634	65.68	Const. T / Remove

Figure 4: Left: two periodic orbits picked on the Morris–Lecar orbit branch, OR1 at $i \approx 100$ ($T \approx 84.0$) and OR2 at $i \approx 200$ ($T \approx 65.7$). Right: the **Special points** table with the picked orbits at the bottom, each with **Const. T** and **Remove**.

Sweep max/min provides a brute-force alternative. It integrates at a grid of parameter values, carrying the end state forward, and plots every local maximum and minimum after the transient. This is the right tool for chaotic or stochastic regimes, where continuation does not apply (Tutorial 4).

Sub- or supercritical? At a Hopf point with eigenvalues $\pm i\omega$, XPP Studio computes the first Lyapunov coefficient

$$\ell_1 = \frac{1}{2\omega}\,\mathrm{Re}\Big[\langle p, C(q,q,\bar{q})\rangle - 2\langle p, B(q, A^{-1}B(q,\bar{q}))\rangle + \langle p, B(\bar{q}, (2i\omega I - A)^{-1}B(q,q))\rangle\Big],$$

where A is the Jacobian, $Aq = i\omega q$, $A^{\mathsf{T}}p = -i\omega p$ and $\langle p, q\rangle = 1$ [5]. The multilinear forms B

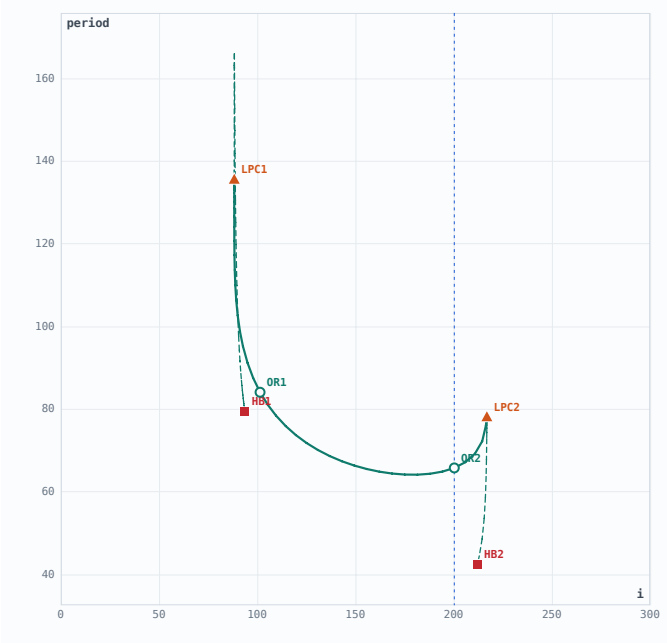


Figure 5: The same diagram with **Plot** set to **period**: the period of the periodic orbits against i, with the special points and the picked orbits.

SPECIAL POINTS 2

PT		I	V	PERIOD	
HB1	sub	93.682	−25.321	79.25	Follow Cycles
HB2	sub	212.06	7.8003	42.33	Follow Cycles

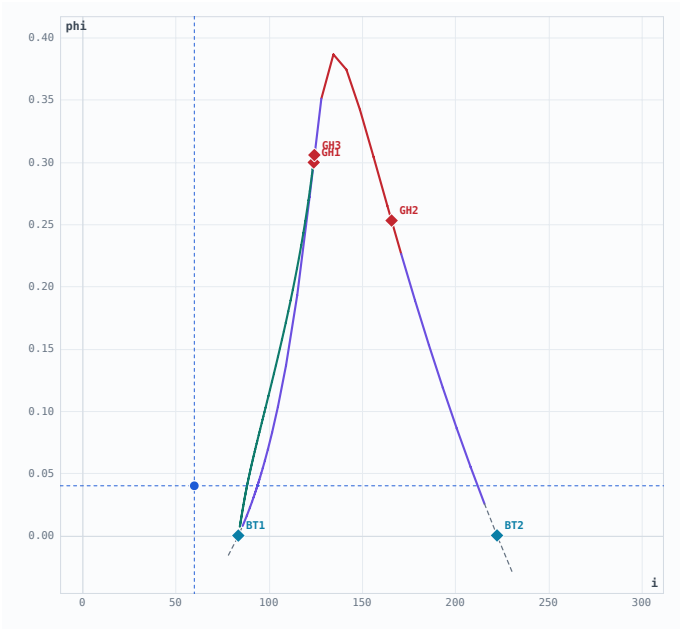
Figure 6: The **Special points** table after continuing equilibria of Morris–Lecar. Both Hopf points are tagged *sub* (subcritical). **Follow** continues a point in two parameters, and **Cycles** continues the periodic orbits born there.

and C (the second and third derivatives of the vector field) are evaluated by finite differences. If $\ell_1 < 0$ the bifurcation is *supercritical*: a small stable orbit appears on the side where the equilibrium is unstable. If $\ell_1 > 0$ it is *subcritical*: an unstable orbit appears on the side where the equilibrium is stable, and the system usually jumps to a distant attractor.

What is detected. For equilibria of flows: folds (LP), Hopf points (HB, tagged sub- or supercritical) and branch points (BP). For periodic orbits: folds of cycles (LPC), period doublings (PD) and torus (Neimark–Sacker) points (TR). For fixed points of maps: LP, PD and Neimark–Sacker (NS). At a PD of cycles, **Double** switches to the period-doubled branch.

6 Tutorial 3: two-parameter continuation

Any special point in the table with a **Follow** button can be continued in a second parameter. The Two-parameter view (5) shows the result, and its inspector panel selects the two parameters



TWO-PARAMETER CONTINUATION

Parameter 1 i

Range 0 to 300

Parameter 2 phi

Range 0 to 0.4

Start LPC1 · i 88.293

Follow curve | Clear curves

fold (LP) | Hopf, supercritical

Hopf, subcritical | neutral saddle

fold of cycles (LPC) | Default colours

CODIMENSION-TWO POINTS 5

PT	I	PHI	ON
BT1	83.618	2.7254e−5	HB
GH1	124.08	0.29974	HB
GH2	165.86	0.25305	HB
BT2	222.44	1.2279e−4	HB
GH3	124.42	0.30569	LPC

Figure 7: Left: two-parameter diagram for Morris–Lecar in (i, ϕ). A single Hopf curve joins HB1 and HB2. It is purple where the Hopf bifurcation is subcritical and red where it is supercritical, and the colour changes at the generalised Hopf (Bautin) points GH1 and GH2. Dashed ends are neutral saddles beyond the Bogdanov–Takens points BT1 and BT2 near $\phi = 0$. The green fold-of-cycles curve from LPC1 ends at GH3, which lies next to GH1. The blue cross-hairs mark the current parameter values. Right: the Two-parameter panel, with a colour picker for each kind of curve and the list of codimension-two points.

and their ranges.

Try it

1. After Tutorial 2, choose the second parameter (here ϕ, the rate of the recovery variable) in the Two-parameter panel.
2. Set the range of ϕ to 0–0.4. Then press **Follow** on HB1 (in the Bifurcation view, or choose HB1 under **Start** and press **Follow curve**). Hopf curves are computed from a bordered system with the bialternate product [4, 5]. Here a single closed arch joins HB1 to HB2, so following HB2 as well would only retrace it.
3. The first Lyapunov coefficient ℓ_1 is computed at every point of the Hopf curve, and the curve is coloured by its sign: purple for subcritical, red for supercritical (Figure 7). Where ℓ_1 changes sign there is a *generalised Hopf* or *Bautin* point, labelled GH. For Morris–Lecar these are at about $(i, \phi) = (124.1, 0.300)$ and $(165.9, 0.253)$, so the Hopf bifurcations are supercritical only near the top of the arch. Bogdanov–Takens (BT), cusp (CP) and zero-Hopf (ZH) points are flagged in the same way. For maps, strong resonances R1 and R2 and fold–flip points are flagged too.

4. Press **Follow** on LPC1 to continue the fold of cycles. Its curve is computed by multiple shooting and is slower, typically 10–20 seconds. Bautin theory predicts that a curve of folds of cycles is born at every GH point, and here the LPC curve runs up the left side of the arch and ends at GH3 $\approx (124.4, 0.306)$. This is the endpoint of the orbit computation and agrees closely with GH1 from ℓ_1, an independent check of both.

5. Each kind of curve has a colour swatch in the panel legend: fold, sub- and supercritical Hopf, neutral saddle, fold of cycles, constant period and period doubling. Click a swatch to choose a new colour. The choice applies at once, is used by **Export SVG**, and is remembered in your browser. **Default colours** goes back to the theme colours.

6. Click a point on any curve to move to those parameter values. Then return to the phase plane or the bifurcation diagram to see the dynamics there.

7. **Curves of constant period.** Pick one or more orbits on the bifurcation diagram (Tutorial 2) and press **Const. T** on one of them, or choose it under **Start** as "OR1 (T=...)" and press **Follow curve**. The orbit is continued in both parameters with its period held fixed: the unknowns are the orbit, i and ϕ, and the extra equation is $T = T_0$. In Figure 8 the two gold curves are the parameter values where Morris–Lecar oscillates with period 84.0 and with period 65.7. Each ends where the orbit shrinks onto an equilibrium at a Hopf point whose period $2\pi/\omega$ equals T_0, so constant-period curves end on the Hopf curve. Integrating at points along a curve gives the same period to within the output spacing.

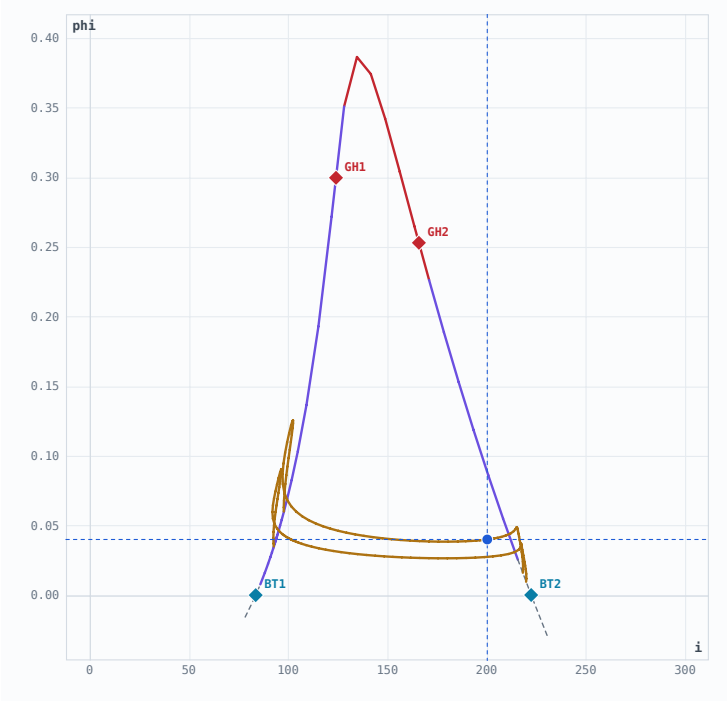


Figure 8: Curves of constant period (gold) for Morris–Lecar in (i, ϕ), traced from the picked orbits OR1 ($T \approx 84.0$) and OR2 ($T \approx 65.7$), together with the Hopf curve, coloured by criticality. Each constant-period curve ends on the Hopf curve, where the frequency of the Hopf point matches the fixed period.

Curves of period doublings of cycles (PDC) are continued the same way, which gives the classical period-doubling tongues of the Rössler system.

7 Tutorial 4: chaos in three dimensions

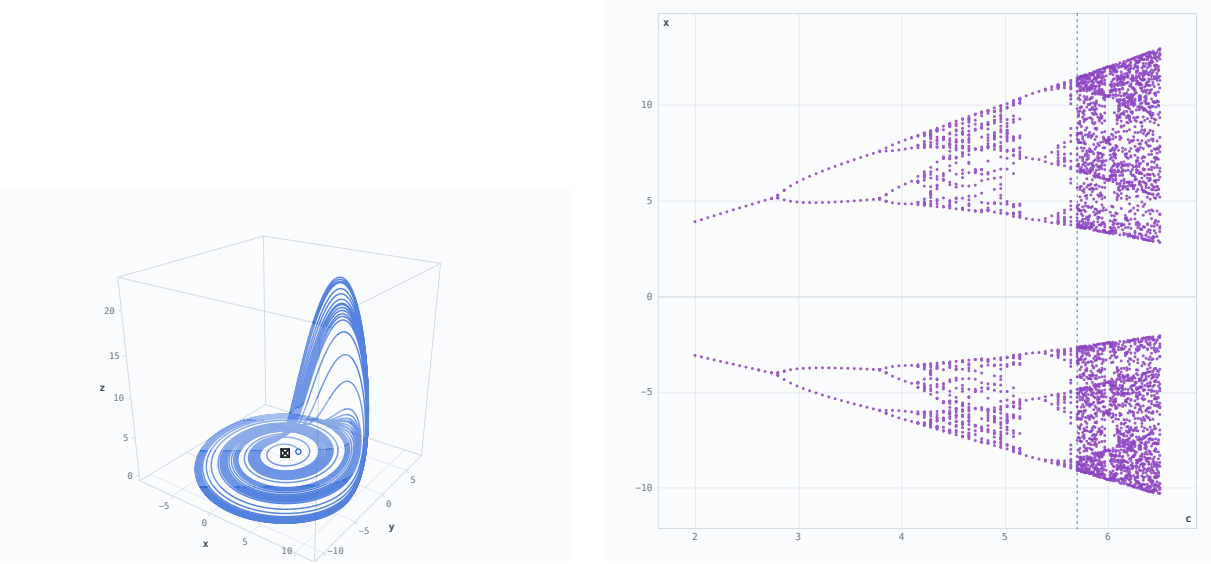


Figure 9: Rössler system at $a = b = 0.2$. Left: the chaotic attractor at $c = 5.7$ in the 3D view. Right: **Sweep max/min** in c, showing the local maxima and minima of x and the period-doubling route to chaos.

Load **Rössler system** [14] and press 2. Drag to rotate the attractor, scroll to zoom, or press **Spin**. The axes are chosen in the toolbar, and any of them can be time.

Try it

1. In the Bifurcation view choose c and press **Sweep max/min** (Figure 9, right). The number of parameter values is set by **Sweep samples** in the Numerics tab.
2. For a continuation version, set $c = 2.5$, integrate onto the stable cycle and press **Limit cycle**. Period doublings are found at $c \approx 2.8324$, 3.8374, 4.1242 and 4.1870. Pressing **Double** at each in turn follows the cascade.
3. **Follow** on a PD point traces its curve in (c, a).

8 Tutorial 5: maps

A model is a map when its equations are written `x(t+1)=...`. Trajectories are then drawn as points, and for a one-variable map the phase plane becomes a cobweb diagram (Figure 10). For maps, **Sweep max/min** keeps up to 250 iterates after the transient, which gives the familiar orbit diagram. Continuation finds folds, period doublings and Neimark–Sacker points of fixed points. The **Hénon map** example shows a fold at $a \approx -0.1225$ and a PD at $a \approx 0.3675$, and the **Delayed logistic map** an NS bifurcation at $a = 2$.

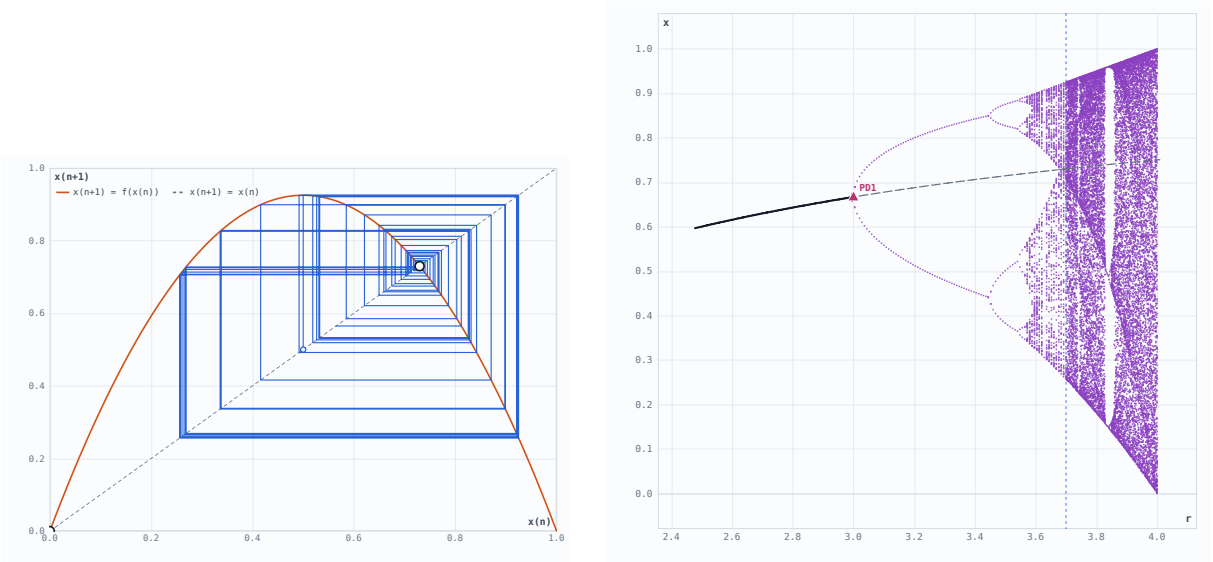


Figure 10: Logistic map $x \mapsto rx(1-x)$. Left: cobweb diagram at $r = 3.7$. Right: orbit diagram from **Sweep max/min** (purple dots), with the continued fixed point and its period doubling PD1 at $r = 3$.

9 Tutorial 6: noise and random switching

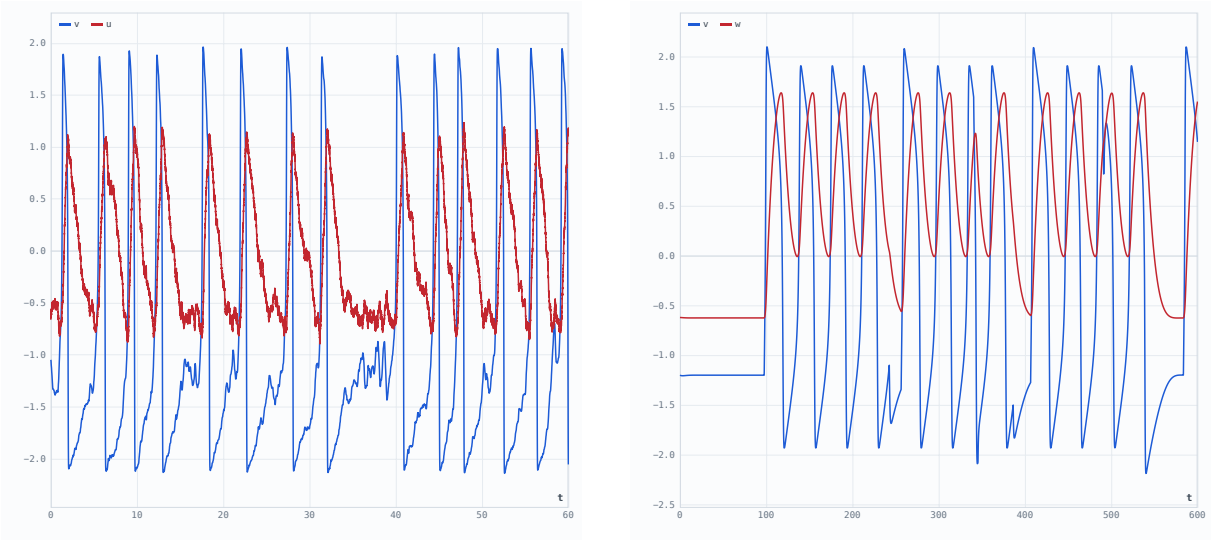


Figure 11: Left: excitable FitzHugh–Nagumo driven by white noise; noise triggers irregular spikes. Right: FitzHugh–Nagumo whose input is a two-state Markov variable, so oscillation switches on and off at random.

Wiener noise. Declaring `wiener w` makes `w` a white-noise source. The equations are integrated by Euler–Maruyama with step `dt`, and over each step `w` is $N(0,1)/\sqrt{dt}$, exactly as in XPPAUT. In **Noisy FitzHugh–Nagumo** the rest state is stable, and noise of amplitude `sig` kicks the system over threshold (Figure 11, left). Drag `sig` to change the firing rate.

Markov variables. **Telegraph-driven FitzHugh–Nagumo** declares

```
markov s 2
{0} {kon}
{koff} {0}
```

a variable that jumps $0 \to 1$ at rate `kon` and $1 \to 0$ at rate `koff`. Rates may depend on the state variables. The input `i0+di*s` switches the cell between rest and oscillation (Figure 11, right).

Seeds. Each stochastic run uses a new random seed, which is reported in the status line. Enter a number in **Noise seed** (Numerics tab), or write `@ seed=7`, to repeat the same noise path, for instance to compare parameter values. Equilibria, nullclines and continuation always use the noise-free system, with Markov states held at the values in the Initial tab.

Power spectrum. In the Time course view the **Spectrum** button replaces the time series by its power spectrum, for the variables selected in the inspector. The run is resampled onto an even grid, the mean is removed, a Hann window is applied and the result is Fourier transformed. The horizontal axis is frequency, in cycles per unit of time (per iteration for maps). The vertical axis is amplitude, scaled so that a sinusoid of amplitude A gives a peak of about A; a peak that falls between two frequency bins can read up to about 15% low. **Log** switches to a logarithmic vertical axis. The legend gives the strongest peak of each variable as a frequency and the corresponding period. Frequencies with fewer than three cycles in the run are left out of this label, though they are still plotted. The frequency resolution is the reciprocal of the length of the run, so integrate for a long time, and set a **Transient** to leave out the approach to the attractor, in order to separate close peaks.

The spectrum of a single noisy run, such as **Noisy FitzHugh–Nagumo**, is ragged (Figure 12, left): at each frequency the estimate fluctuates by an amount comparable to its value. **Smooth** cuts the run into overlapping segments, each a quarter of its length, and averages their spectra (Welch's method). The result is a smooth curve whose broad peak lies near the typical spiking rate (Figure 12, right), at the price of a frequency resolution four times coarser. The same button helps with chaotic signals: for the **Lorenz equations** the spectrum is broadband, with a clear peak in z near $f \approx 1.3$. For a clean periodic orbit leave **Smooth** off, so that the peaks stay sharp.

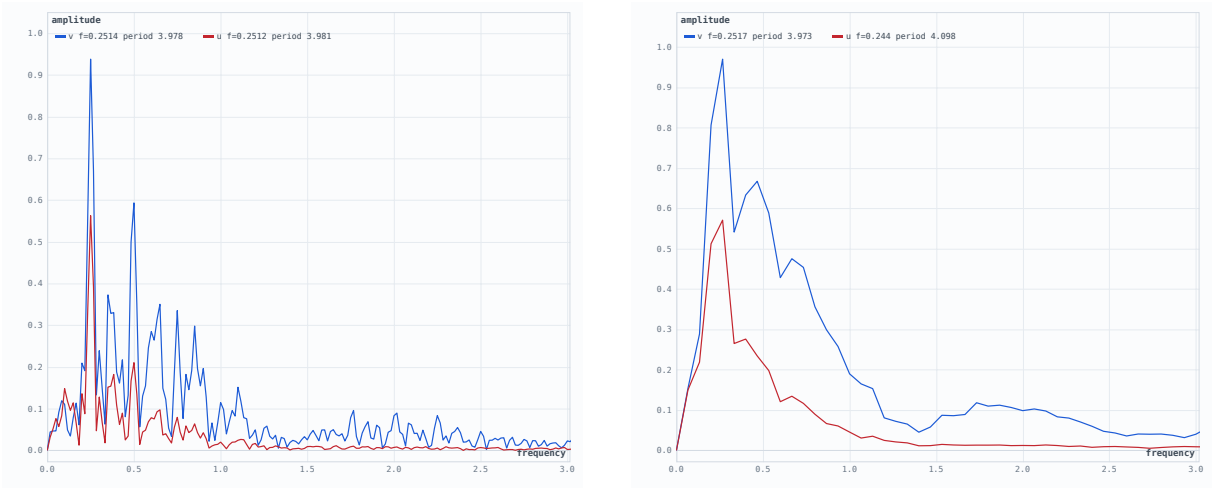


Figure 12: Power spectrum of v and u in **Noisy FitzHugh–Nagumo** for a single run (`@ seed=7`, so that both panels use the same run). Left: the raw spectrum is ragged. Right: with **Smooth**, the average over overlapping segments gives a smooth curve with a broad peak near the spiking rate, at four times coarser frequency resolution.

10 Tutorial 7: stiff systems

The **Stiff van der Pol** example has $\mu = 1000$: relaxation oscillations whose slow and fast time scales differ by a factor of about 10^6. With `@ meth=stiff` the Rosenbrock 2(3) solver integrates

a period ($T \approx 1614.4$) in a few thousand steps, where an explicit method needs hundreds of thousands. When the stiff method is selected, periodic-orbit continuation also uses it for its shooting segments, so the period can be continued down in μ.

11 Tutorial 8: arrays and Volterra equations

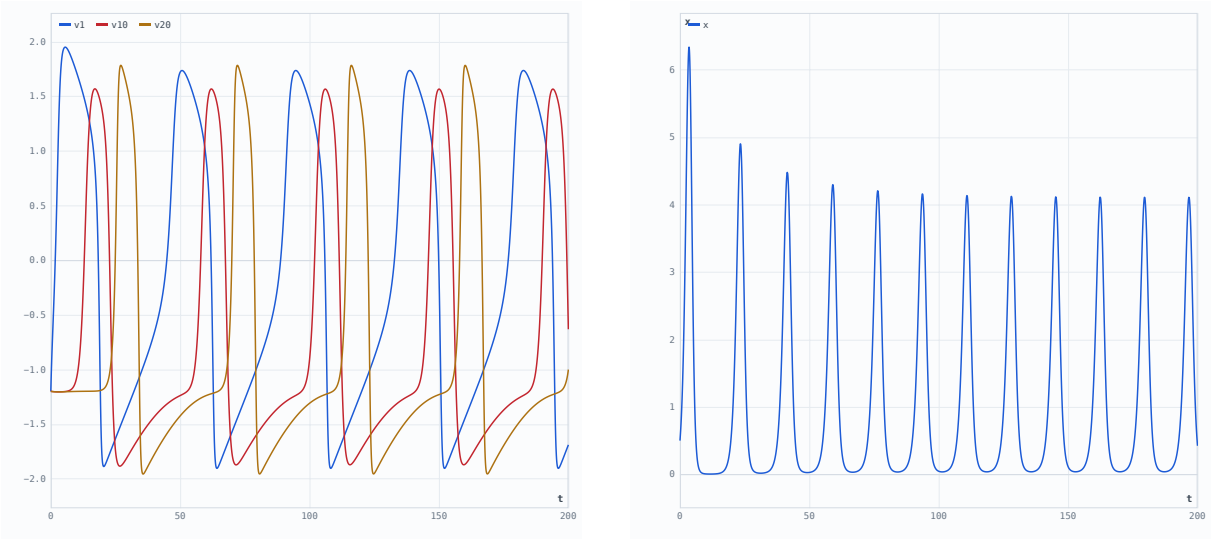


Figure 13: Left: a chain of 20 FitzHugh–Nagumo cells written with arrays. Spikes start at cell 1 and arrive later at cells 10 and 20. Right: logistic growth with a gamma-distributed delay, written as a Volterra convolution. At $r\tau = 2.5$ the equilibrium is unstable and the population oscillates.

Arrays. XPPAUT's arrays are a text macro. The **Chain of 20 FitzHugh–Nagumo cells** example is written as

```
%[1..20]
v[j]'=v[j]-v[j]^3/3-w[j]+i1*heav(1.5-[j])+d*(v[j+1]-2*v[j]+v[j-1])
%
w[1..20]'=eps*(v[j]+a-b*w[j])
v0=v1
v21=v20
init v[1..20]=-1.2
init w[1..20]=-0.62
```

The block between `%[1..20]` and `%` is repeated for $j = 1, \ldots, 20$. Every `[expr]` is replaced by its integer value, so `v[j-1]` becomes `v3` when $j = 4$, and `[j]` on its own becomes the number j. A single line containing `[1..20]` is repeated the same way. The load message reports how many lines the file expanded to. The expanded model is an ordinary 40-dimensional system, so its equilibria, eigenvalues (computed by the QR algorithm) and continuation all work as usual. In Figure 13 (left) the time course shows v_1, v_{10} and v_{20}; choose them with the chips in the inspector.

Space-time plot. For a chain or lattice it is often clearer to see all the cells at once than as overlapping curves. A model with a row of indexed variables, such as $v_1, \ldots, v_{20}$, has a **Space-time** button in the Time course view; it does not appear for other models. Arrays are a text macro that the program does not remember after loading, so a row is recognised from the names alone: at least six variables with the same base name and consecutive integer suffixes. The button replaces the time series by an image in which time runs along the horizontal axis, the

cell number up the vertical axis, and the colour gives the value (Figure 14). A pulse started at cell 1 is a stripe running up the picture, and its slope is the speed at which the pulse propagates.

- *Choosing the rows.* The chips in the inspector, now headed **Space-time variables**, select the variables drawn (Figure 15). Click a chip to add or remove it, shift-click a second chip to select the whole range from the last chip clicked, or use **All** and **None**. The menu above the plot offers ready-made sets: each row of variables ($v_1, \ldots, v_{20}$ or $w_1, \ldots, w_{20}$) and all the variables. This selection is kept apart from the variables chosen for the time series and the spectrum.
- *Rows.* Rows come in the order of the variables. If the chosen variables are an unbroken run from one family, the vertical axis shows the cell number. Otherwise, with gaps or a mixture of families, the rows are packed together with no blank bands and each is named at the left edge (Figure 16).
- *Colour.* The colour bar to the right of the plot gives the range, by default the smallest and largest value of the chosen rows over the run. When several variables share one scale, those with small values look washed out (Figure 16, left). Type values into the **colour min** and **max** boxes to set the range by hand, as in the right panel, and press **Auto** to return to the automatic range.
- Hovering gives the time, the variable and its value, and **Export SVG** saves the picture, including the colour bar.

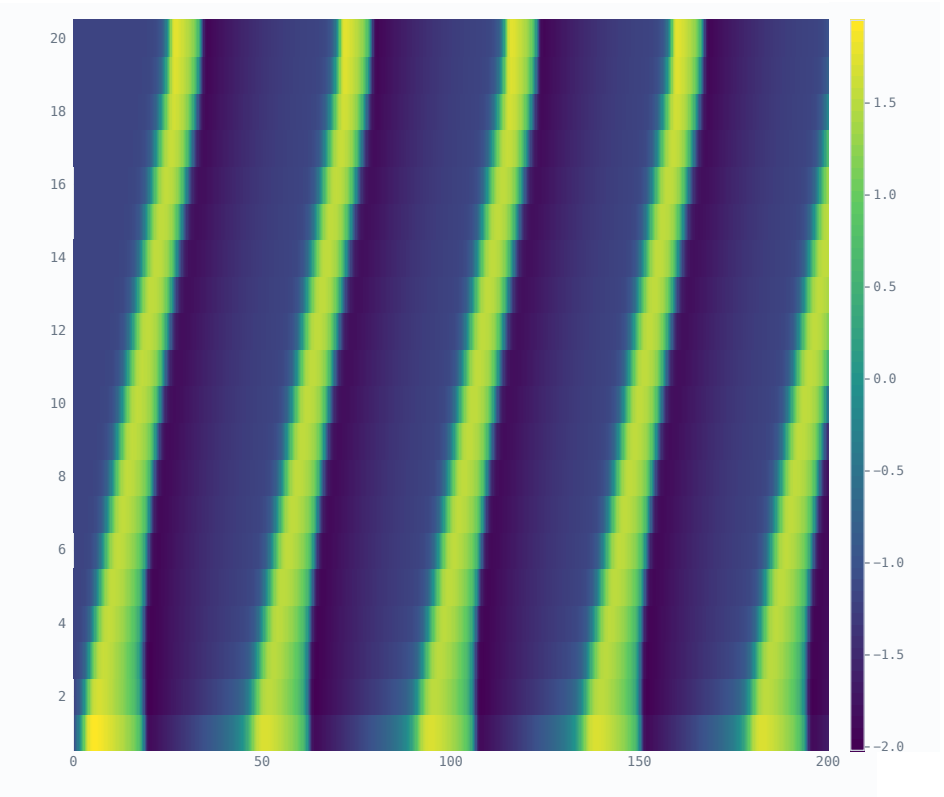


Figure 14: Space-time plot of $v_1, \ldots, v_{20}$ for the chain of FitzHugh–Nagumo cells. Time runs along the horizontal axis, the cell number up the vertical axis, and the colour gives v_j, with its range in the colour bar. Each stripe is a pulse started at cell 1 and travelling up the chain; compare the time courses of v_1, v_{10} and v_{20} in Figure 13.

Volterra integrals. XPP Studio supports XPPAUT's integral terms:

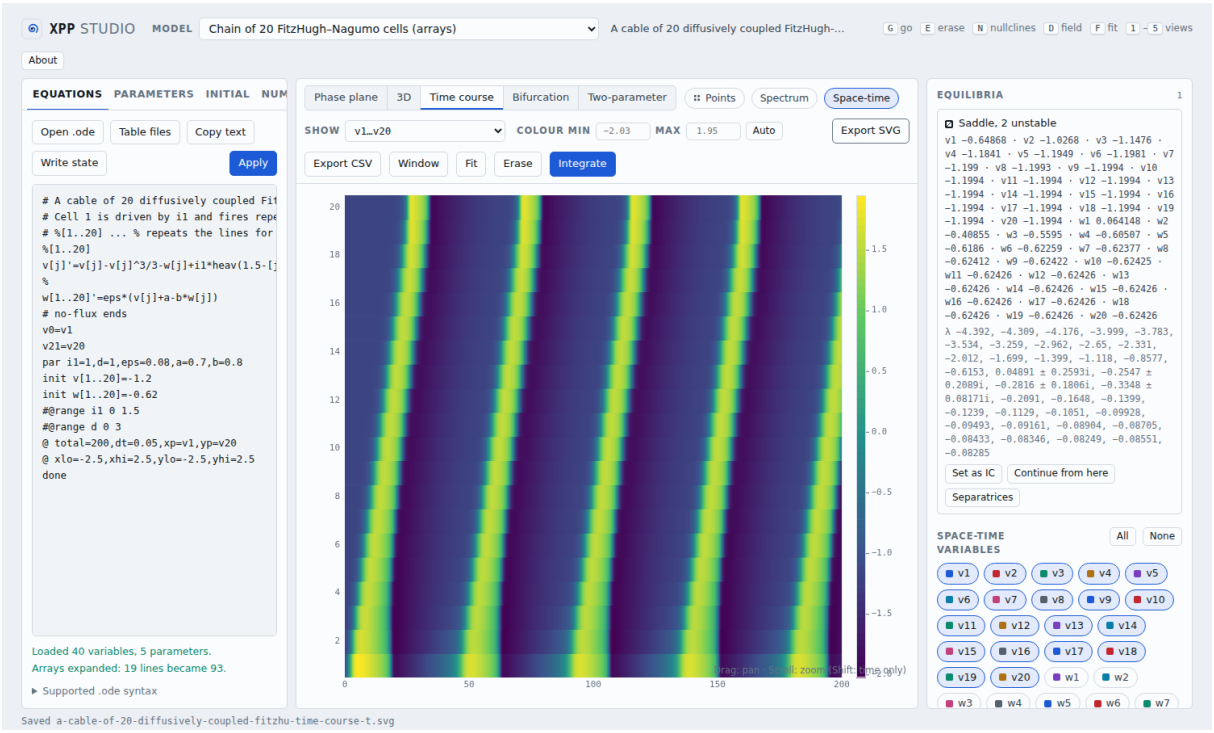


Figure 15: The Time course view in space-time mode for the chain. Above the plot are the **Space-time** button, the menu of sets of rows, the **colour min** and **max** boxes and **Auto**. The colour bar is to the right of the plot, and the chips at the bottom right choose the rows.

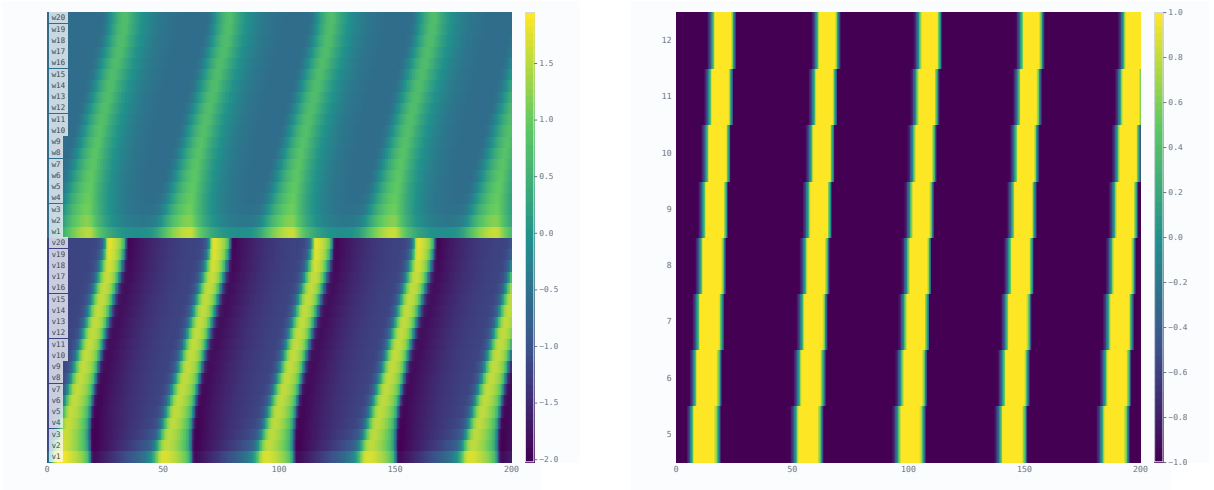


Figure 16: Choosing the rows. Left: all 40 variables, $v_1, \ldots, v_{20}$ below and $w_1, \ldots, w_{20}$ above; the w variables are small, so on a common colour scale they look washed out. Right: the cells $v_5, \ldots, v_{12}$, chosen by shift-click, with the colour range set by hand to -1 to 1.

- `int{K(t,t',x)}` $= \int_0^t K(t,t',x(t'))\,dt'$, where the variables inside are evaluated at t';
- `int{K#x}` $= \int_0^t K(t-t')\,x(t')\,dt'$, a convolution with the kernel written as a function of t;
- `int[mu]{...}` adds the weakly singular factor $(t-t')^{-\mu}$, $0 \le \mu < 1$;
- `u(t)=...` or `volterra u=...` defines a variable by an integral equation.

The **Logistic growth with distributed delay** example is

```
x'=r*x*(1-int{t*exp(-t/tau)/tau^2#x})
par r=1,tau=2.5
@ total=200,dt=0.05,vmaxpts=1000
```

The kernel $K(s) = s\,e^{-s/\tau}/\tau^2$ has mean delay 2τ. Linearising about $x = 1$ gives $\lambda(1+\lambda\tau)^2 + r = 0$, so the equilibrium loses stability in a Hopf bifurcation at $r\tau = 2$. Sweeping τ with **Sweep max/min** shows the oscillation appear.

Integro-differential equations are solved by a fixed-step method: the trapezoid rule for the differential part and product-trapezoid quadrature [8] for the integrals. The quadrature weights are exact for the $(t-t')^{-\mu}$ factor, and the implicit terms are found by fixed-point iteration. The error falls as dt^2, or as $dt^{2-\mu}$ with a singular kernel. The method reproduces $u(t) = e^t$ for $u = 1 + \int_0^t u$, and the solution $e^{\pi t}\,\mathrm{erfc}(\sqrt{\pi t})$ of the Abel equation `u(t)=1-int[.5]{1#u}`. It also agrees with the equivalent three-variable ODE for the gamma kernel. Only the last `vmaxpts` steps are remembered (default 4000). Because the state of such a system is its whole history, equilibria, nullclines and continuation are switched off for these models. Trajectories and sweeps work normally.

Periodic variables and tables. Two further XPPAUT features are worth knowing:

- `@ fold=th` keeps an angle in $[0, 2\pi)$ (or $[0,$ `tor_per`$)$). Plots break the trajectory where it wraps, and continuation measures distances modulo the period; see the **Driven pendulum** example.
- Tables (**Phase oscillators with a tabulated H**) are interpolated linearly. They can be sampled from an expression or read from a data file.

12 Tutorial 9: delay equations

A delay-differential equation uses past values of its variables. In XPP Studio, as in XPPAUT, `delay(x,tau)` stands for $x(t-\tau)$, where `x` must be a variable with an equation and `tau` is any expression; it may depend on the state. Load **Delayed logistic equation**, Hutchinson's model [12]:

```
x'=r*x*(1-delay(x,tau))
aux xlag=delay(x,tau)
par r=1.8,tau=1
x(0)=0.5+0.2*cos(3*t)
```

History. A delay equation needs its state on a whole interval $[-\tau, 0]$, not only at $t = 0$. Writing `x(0)` as a formula in `t` gives that history, and its value at $t = 0$ becomes the initial condition. Without a formula, the history is held constant at the initial value. The formulas also appear in the **Initial** tab under **History for t < 0** (Figure 18), where they can be changed or cleared. Clicking the phase plane changes only the values at $t = 0$. The auxiliary variable `xlag` makes $x(t-\tau)$ available for plotting, so the phase plane in Figure 17 shows $x(t-\tau)$ against $x(t)$, a common way to draw the attractor of a scalar delay equation.

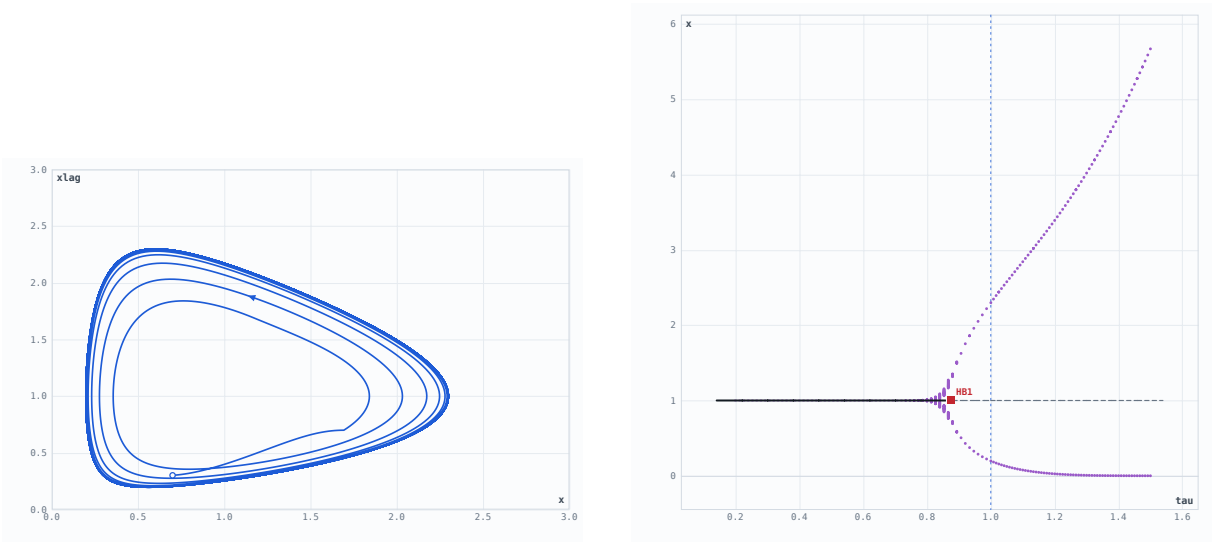


Figure 17: Hutchinson's delayed logistic equation with $r = 1.8$. Left: $x(t-\tau)$ against $x(t)$ for $\tau = 1$, starting from the history $0.5 + 0.2\cos 3t$ and settling onto a periodic orbit. Right: bifurcation diagram in τ. The equilibrium $x = 1$ (black) loses stability at the Hopf point HB1, $\tau \approx 0.874$, close to the exact value $\pi/(2r) = 0.873$. The purple dots, from **Sweep max/min**, show the oscillation that grows from it.

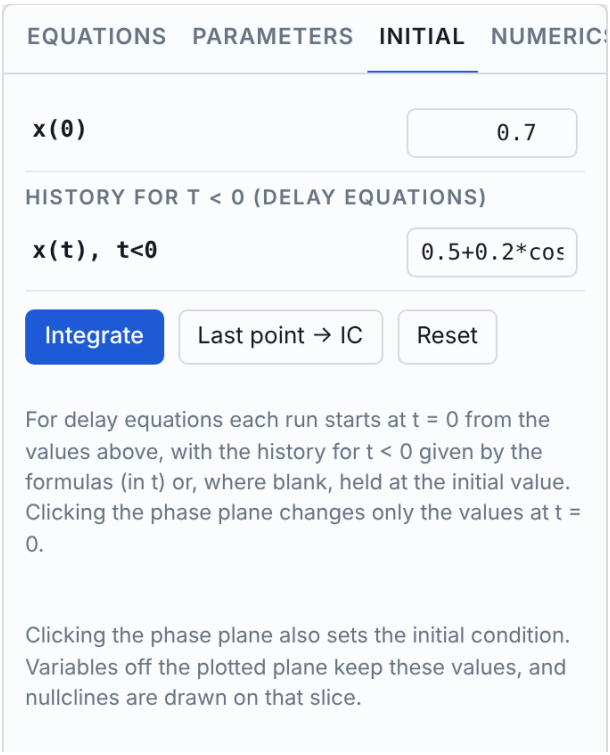


Figure 18: The Initial tab for a delay equation: the value at $t = 0$ and, below it, the history for $t < 0$ as a formula in t.

Integration. Delay equations run forward from $t = 0$ with fixed-step fourth-order Runge–Kutta (adaptive methods fall back to it). Every step is stored, and delayed values are found by cubic Hermite interpolation of the stored solution, or from the history when $t - \tau < 0$. For $x' = -x(t-1)$ with $x = 1$ on $[-1, 0]$, whose exact solution is piecewise polynomial, the computed $x(2)$ and $x(3)$ agree with the exact values to rounding error.

Stability and continuation. Equilibria of a delay equation are those of the system with the delays removed, but their stability is not. It is set by the roots of the characteristic equation

$$\det\Big(\lambda I - A_0 - \sum_k B_k\, e^{-\lambda \tau_k}\Big) = 0,$$

where A_0 and B_k are the derivatives with respect to the current and the delayed values. There are infinitely many roots, but only finitely many lie to the right of any vertical line. XPP Studio approximates the rightmost ones by Chebyshev collocation of the solution operator [11] and then polishes them with Newton's method. The equilibrium list then reports, for example, *Unstable, 2 roots with Re* $\lambda > 0$ and lists the rightmost roots. **Continue equilibria** uses the same roots, so Hopf points are those of the delay equation.

Try it

1. Load **Delayed logistic equation** and press G. With $r\tau = 1.8 > \pi/2$ the solution settles onto a periodic orbit (Figure 17, left). Change the history formula in the Initial tab and integrate again.
2. The inspector lists two equilibria. $x = 0$ has the single root $\lambda = r$. $x = 1$ has a complex pair $\lambda \approx 0.097 \pm 1.63i$ in the right half-plane.
3. In the Bifurcation view choose `tau`, a range of 0.2 to 1.5 and **Step ds** 0.02, then press **Continue equilibria**. A Hopf point HB1 is found at $\tau \approx 0.874$, against $\pi/(2r) = 0.8727$. **Sweep max/min** adds the oscillation amplitude (Figure 17, right).
4. Load **Mackey–Glass equation** [13] (Figure 19). With $\tau = 17$ the solution is chaotic; the file uses `@ trans=300` to drop the transient. Continuation in τ finds Hopf points at $\tau \approx 4.71$ and 20.98. Linearising by hand gives $\lambda = -0.1 - 0.4\, e^{-\lambda\tau}$, with Hopf points at $\tau = 4.708$ and 20.93.

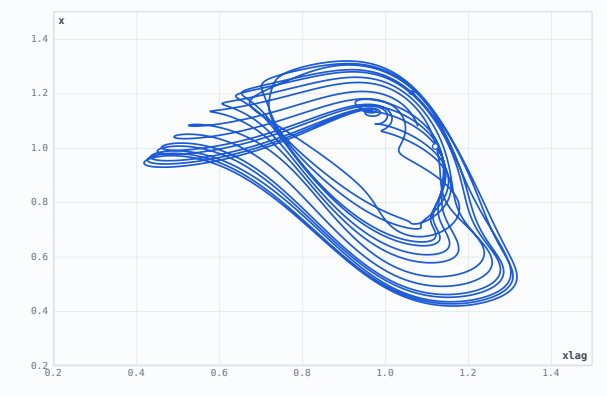


Figure 19: The chaotic Mackey–Glass attractor at $\tau = 17$, drawn as $x(t)$ against $x(t-\tau)$ after a transient of 300.

Periodic-orbit and two-parameter continuation, the direction field and the derivative readout are switched off for delay equations, because the state at time t is a whole function rather than a point.

13 Tutorial 10: events and global flags

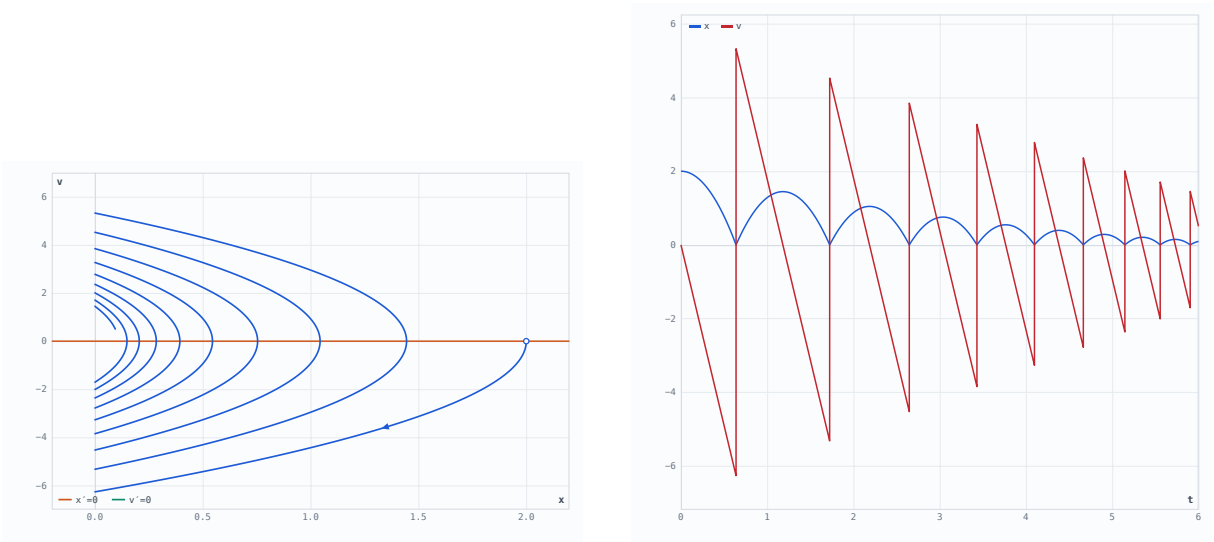


Figure 20: The bouncing ball. Left: velocity against height. Each parabola is one flight, and the line breaks at each event instead of joining the points before and after it. Right: height x (blue) and velocity v (red) against time, with the jump in v at each bounce.

Many models change discontinuously when a threshold is reached: a neuron fires and resets, a ball bounces, a switch flips. XPPAUT describes such events with *global flags*,

```
global sign condition {name1=expr1;name2=expr2;...}
```

The assignments are made when `condition` crosses zero from negative to positive (`sign`$= 1$), from positive to negative (-1), or in either direction (0). All the right-hand sides are evaluated first, then all the assignments are made together. The names may be state or Markov variables, and `arret=1` stops the integration.

How events are located. After every step XPP Studio checks each condition. If one has crossed zero in its direction, the crossing time is found on the cubic Hermite interpolant of that step, by regula falsi on the condition. The point just before the event and the point just after the assignments are both stored, at the same time, and integration continues from there. This works with every integrator, including the adaptive and stiff ones, and with maps, where the assignments are applied to the new iterate.

Try it

1. Load **Bouncing ball**:

   ```
   x'=v
   v'=-g
   global -1 x {x=0;v=-e*v}
   ```

   Press G. The status line reports the number of events. The first two bounces occur at $t = \sqrt{2x_0/g} = 0.6386$ and $t = 0.6386\,(1 + 2e) = 1.7242$, and the computed times agree to six decimal places. Each flight is a parabola in the phase plane (Figure 20). Lower the restitution e with its slider and watch the bounces die away faster.

2. Load **Integrate-and-fire neuron**:

   ```
   v'=(-v+i-w)/tau
   ```

```
w'=-w/tauw
global 1 v-1 {v=0;w=w+b}
```

Each time v reaches threshold it is reset and the adaptation variable w is kicked up, so the firing rate slows until it settles (Figure 21). With $b = 0$ the interspike interval is $\tau \ln(i/(i-1))$, and the computed value agrees to six or more digits.

3. Add `global 1 t-50 {arret=1}` to a model to stop every run at $t = 50$.

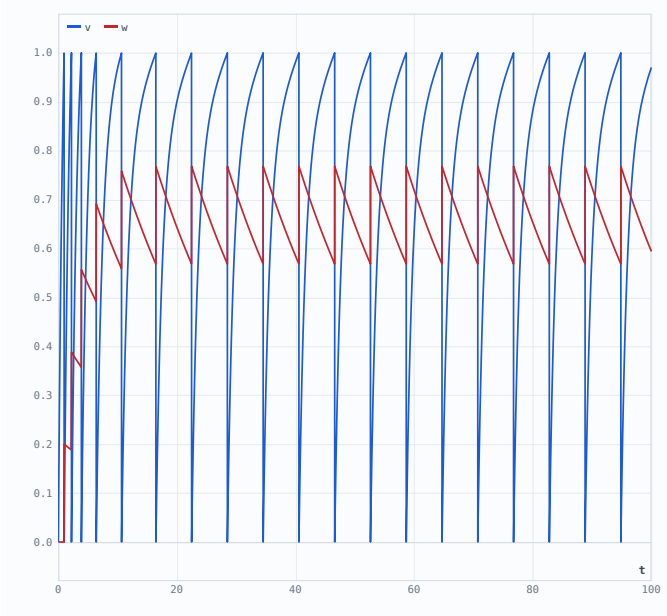


Figure 21: Integrate-and-fire neuron with adaptation: v (blue) is reset to 0 at each threshold crossing, while the adaptation current w (red) builds up and lengthens the interspike interval.

Equilibria, nullclines and equilibrium continuation ignore flags. Periodic-orbit continuation is switched off for models with flags; use **Sweep max/min** for the long-term behaviour instead.

14 Tutorial 11: boundary value problems

A boundary value problem asks for a solution of the differential equations that satisfies conditions at both ends of an interval, not only at the start. XPPAUT solves such problems by shooting, and so does XPP Studio: Newton's method adjusts all the initial values until every boundary condition is zero.

Boundary conditions. A line `b` *expression* (also written `bndry` or `bdry`) says that the expression is zero. In it an unprimed name is the value of that variable at the left end, $t = t_0$, and a primed name is its value at the right end, $t = t_0 + T$, where T is the total time. So `b u-1` and `b u'-2` say $u(0) = 1$ and $u(T) = 2$. The time `t` may be used in a condition and has the value t_0. There must be one condition for each variable: an unknown parameter is declared as a variable with zero derivative, such as `lam'=0`, so that it becomes one of the unknowns (see below). Given initial values, the equations are integrated from t_0 to $t_0 + T$ with the current method and step, the conditions are evaluated, and Newton's method, with a finite-difference Jacobian and a damped step, repeats until the largest condition is below 10^{-8}.

Bratu's problem. The **Bratu boundary value problem** example is $u'' = -\lambda e^u$ with $u(0) = u(1) = 0$:

```
par lam=1
u'=up
up'=-lam*exp(u)
b u
b u'
init u=1
#@irange up 0 2
@ total=1,dt=.01
```

The two conditions are $u(0) = 0$ and, because of the prime, $u(1) = 0$. There are two variables, u and $u' =$ `up`, so the unknowns are $u(0)$ and $u'(0)$. The starting guess $u(0) = 1$, $u'(0) = 0$ does not satisfy either condition.

Try it

1. Load **Bratu boundary value problem** and press G. The run from the guess misses both conditions. Open the **Initial** tab: the **Boundary conditions** list shows the residual of each condition for the current trajectory (here 1 and a nonzero value for `b u'`).
2. Press **Solve BVP** above the plot. Newton's method converges in four iterations, the initial values become $u(0) = 0$ and $u'(0) = 0.5494$, and the residuals fall to about 10^{-14} (Figure 23, left). The status line reports the residual and the number of iterations.
3. With **Solve BVP** on, the problem is solved again whenever a parameter, an initial value or a numerics setting changes. Open the **Parameters** tab and drag `lam`: the solution follows, each solve starting from the previous one. Type larger values into the box, since the slider only covers 0 to 2. If Newton fails from a big jump, the program follows the solution from the old parameter value to the new one in smaller steps, which is what makes a long jump work when a solution exists.
4. Type 3.6 for `lam`. There is no solution beyond $\lambda \approx 3.5138$, and the status line says so, adding how far the solution could be followed (about 3.51). The initial values stay at those of the last solution.
5. Return to $\lambda = 1$. In the **Initial** tab type 9 for `up(0)` and press Enter. The typed value is a new starting guess, and Newton converges to the *upper* solution, $u'(0) = 10.85$ (Figure 23, right). The same equations have two solutions for $\lambda < 3.5138$.
6. Press **Solve BVP** again to return to initial value problems. The initial values stay where they were.

Sliders for initial values. In a boundary value problem the initial values are the starting guess, and it helps to move them by hand. A comment line `#@irange up 0 2` gives the initial value of `up` a slider in the **Initial** tab, in the same way as `#@range` does for a parameter. Dragging it reruns the current trajectory from the new value, and the residuals update as you drag, so you can steer a guess until the residuals are small. If **Solve BVP** is on, releasing the slider solves from that guess. XPPAUT reads the line as a comment, so files remain portable, and **Write state** keeps the slider ranges up to date.

Continuing the solution. In the Bifurcation view a model with boundary conditions has a **Continue BVP** button in place of the equilibrium, sweep and limit-cycle buttons. Choose the parameter and its range (here `lam`, 0 to 4), and under **Plot** choose a variable or an L^2 norm, and press the button. The solution at the current parameter is the starting point. The solutions form

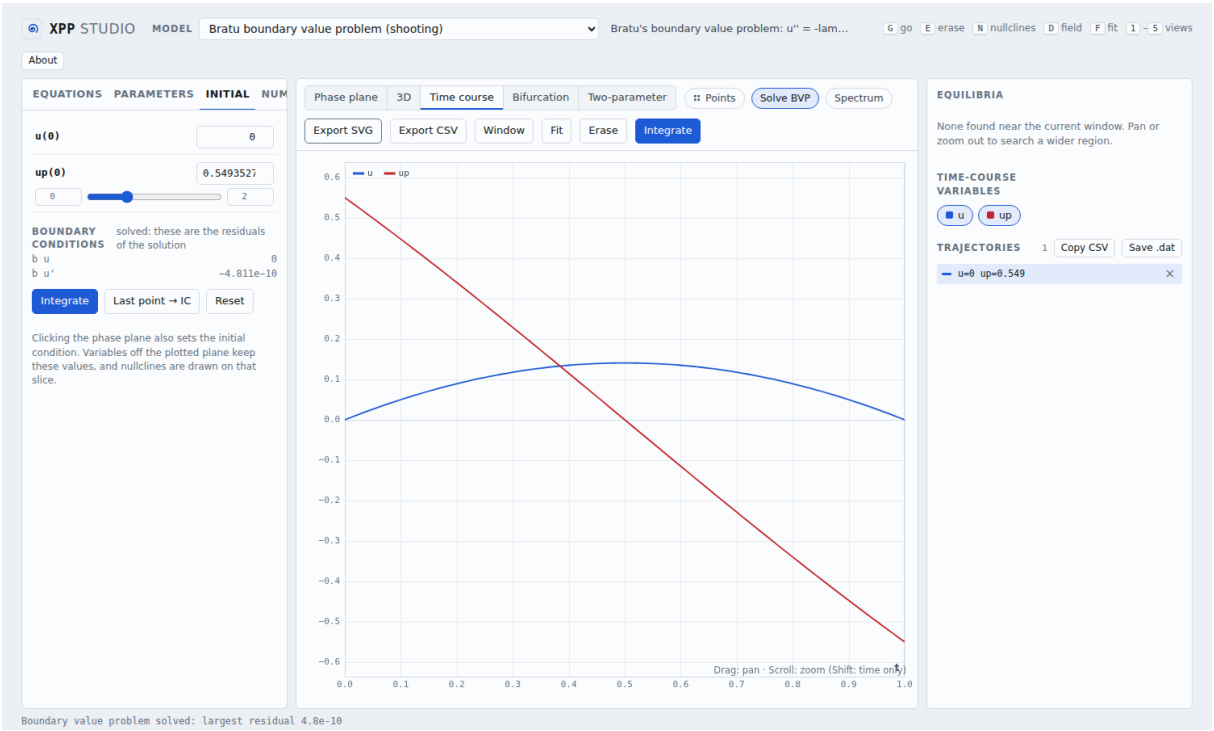


Figure 22: The Bratu example in solve mode. **Solve BVP** is on, the **Initial** tab shows the slider for $u'(0)$ and the boundary residuals of the solution, and the plot shows u and u'.

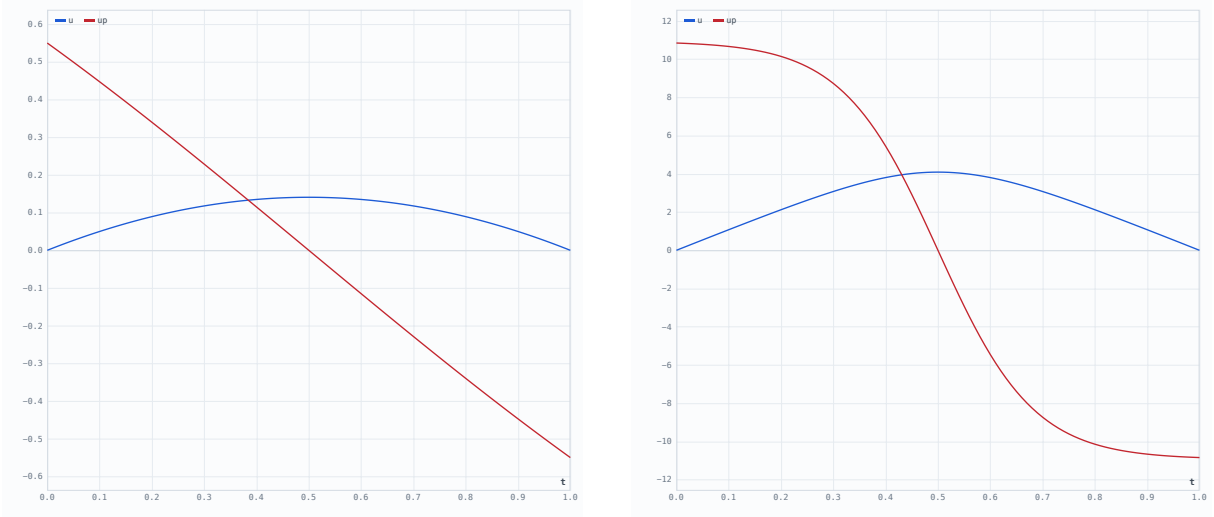


Figure 23: The two solutions of Bratu's problem at $\lambda = 1$: u (blue) and u' (red). Left: the lower solution, $u'(0) = 0.5494$, found from the default guess. Right: the upper solution, $u'(0) = 10.85$, found by typing 9 as the starting value of $u'(0)$.

a curve in the space of initial values and the parameter, which is followed in both directions by pseudo-arclength continuation [6]: a tangent predictor and a Newton corrector, with the initial values and the parameter treated alike, so the branch turns round folds. The plotted quantity is the maximum and minimum of the chosen variable over the solution, as for limit cycles, or the L^2 norm $\left(\int_{t_0}^{t_0+T} u^2\,dt\right)^{1/2}$ of a variable.

For Bratu's problem the branch starts on the lower solutions, turns round the fold `LP1` at $\lambda = 3.5138$ and continues on the upper solutions, which grow without bound as $\lambda \to 0$ (Figure 24). The fold is listed in the **Special points** table, where it agrees with the exact value $3.5138307\ldots$ to eight digits. Clicking a point of the branch, or a row of the table, moves to that parameter value and loads the solution there, so you can then look at it in the Time course view. **Clear diagram**, **Export SVG**, **Export CSV** and **Export .dat** work as for other branches.

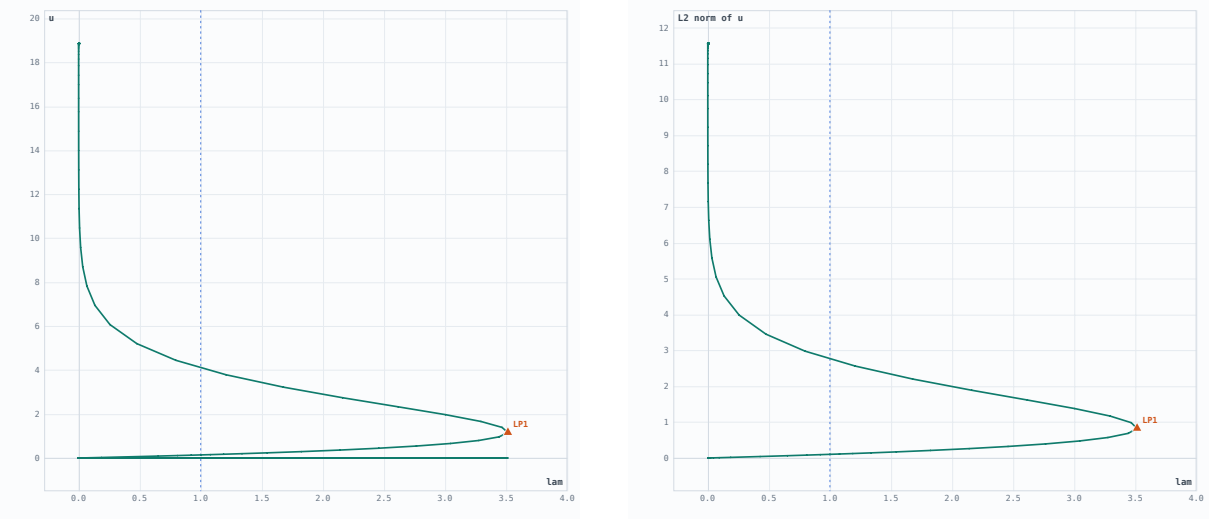


Figure 24: Continuation of Bratu's problem in λ. Left: the maximum of u (the line along the axis is its minimum, which is zero). Right: the L^2 norm of u. The branch turns round the fold `LP1` at $\lambda = 3.5138$ from the lower to the upper solutions; the dashed vertical line marks the current $\lambda = 1$.

Unknown parameters. The **Eigenvalue boundary value problem** example finds the eigenvalue λ of $u'' = (kt - \lambda)u$ with $u'(0) = u'(1) = 0$, normalised so that $\int_0^1 u^2\,dt = 1$:

```
par k=1
u'=up
up'=(k*t-lam)*u
n'=u^2
lam'=0
b n
b up
b up'
b n'-1
init lam=10.4,u=1
@ total=1,dt=.01
```

The variable `n` accumulates $\int u^2\,dt$, and the unknown `lam` is a variable with zero derivative. There are four variables and four conditions: $n(0) = 0$, $u'(0) = 0$, $u'(1) = 0$ and $n(1) = 1$. With **Solve BVP** on, `lam` converges from 10.4 to 10.3762. Setting `k` to 0 gives $\lambda = \pi^2 = 9.8696$, and with **Solve BVP** on, dragging `k` makes `lam` follow. Continuing in `k` with `lam` under **Plot** draws the eigenvalue as a function of `k`.

A singular end. The **Rotating spiral wave** example (`gberg.ode`) is a boundary value problem for a rotating spiral wave of a two-dimensional equation, with three dynamic variables and a free shooting parameter `omeg`, again a variable with zero derivative. The equations are singular at $t = 0$, so the file starts at $t_0 = 0.01$ with `@ t0=.01,total=.99`. The first condition, `bndry a-t*ap`, uses `t`, which is the start time. Solving gives $\omega = -0.1988$ at $d = 0.177$, and continuing in `d` with `omeg` under **Plot** shows ω rising from -0.41 at $d = 0.05$ to nearly zero at $d \approx 0.294$. There the amplitude $a(0)$ vanishes. The equations are odd in a, so the branch continues through the vanishing amplitude as the mirror solution with $a < 0$, which has the same ω, and the turning point in d is marked as a fold.

What is and is not covered.

- Boundary value problems are for ordinary differential equations. Maps, delays, Volterra integrals, Wiener noise and Markov variables give an error when there are boundary conditions. A file with a different number of conditions than variables loads with a warning, and **Solve BVP** then explains the mismatch.
- If the method is Dormand–Prince or one of the stiff methods, the shooting integrations use fixed-step Runge–Kutta 4 with step `dt`, because the step selection of an adaptive method makes the finite-difference Jacobian noisy. The status line says when this happens. Shooting loses accuracy when solutions grow exponentially across the interval; a smaller `dt` or a shorter interval can help.
- A continuation stops at the end of the parameter range, at the point limit (**Nmax**, at most 1000 points in each direction), when the solution grows beyond the **Bound**, or when Newton fails and the step becomes too small. The status line says which. **Esc** skips to the other direction and Shift+Esc stops it.
- Folds are found, but not branch points, and no stability is assigned. Two-parameter continuation of folds is not available for boundary value problems.

15 Numerical settings

The **Numerics** tab (Figure 25) collects the integration and continuation parameters. Changes to the integration settings rerun every trajectory on screen.

Integration.

Method As in Section 3.

Total time The duration of each run; for maps, the number of iterations.

Start time Each run goes from t_0 to t_0+ the total time (`@ t0=`); the default is 0. It is used when the equations are singular at $t = 0$, as in a boundary value problem, and applies to ordinary differential equations only.

Transient As in XPPAUT, the run still lasts the total time, but only $t \geq$ the transient is kept (`@ trans=`). Use it to look at attractors without the approach to them. Manifolds and the single periods loaded from continuation are never cut.

dt The step for fixed-step methods, and the largest step for adaptive ones.

Tolerance Applies to the adaptive methods.

Bound Integration stops when any variable exceeds this value.

Direction Forward or backward in time.

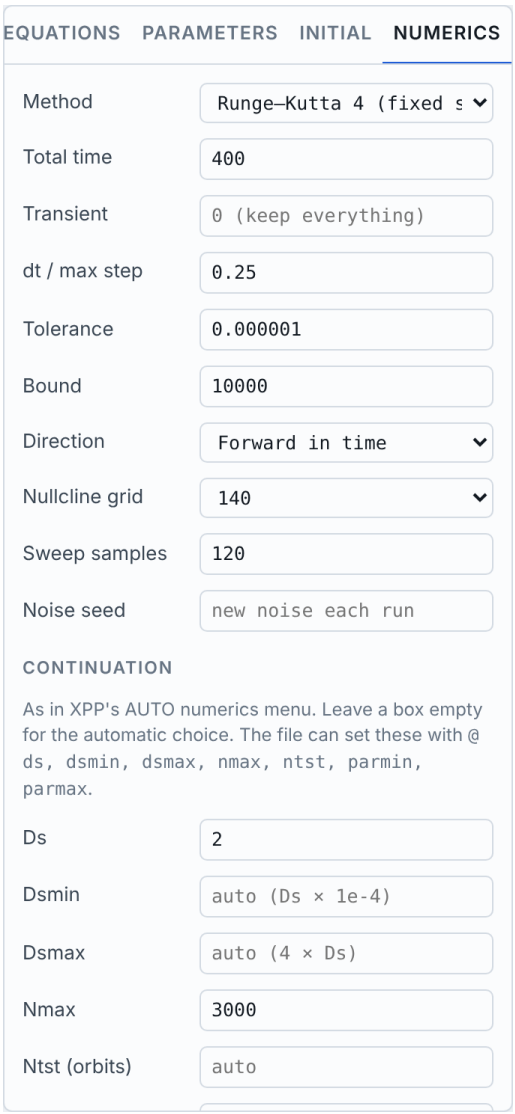


Figure 25: The Numerics tab: integration settings at the top, continuation settings below.

Nullcline grid Sets the resolution of the nullclines.

Sweep samples Sets the number of parameter values in **Sweep max/min**.

Continuation. These settings correspond to the entries of XPPAUT's AUTO numerics window and can be set in the file with `@ ds, dsmin, dsmax, nmax, ntst, parmin, parmax`. Leave a box empty to use the automatic choice.

Setting	Effect
Ds	initial arclength step for equilibrium branches (same as **Step ds**)
Dsmin, Dsmax	smallest and largest steps; a branch stops if the step falls below Dsmin
Nmax	maximum number of points in each direction of an equilibrium branch
Ntst (orbits)	number of shooting intervals per periodic orbit (automatic: 20 for up to 3 variables, 14 up to 5, otherwise 10)
Ds, Dsmax, Nmax (orbits)	the same for periodic-orbit branches and curves of cycles
Nmax (2-par), Nmax (2-par cycles)	point limits for two-parameter curves

If a branch stops early, the status line says why: for example, it reached the point limit, the corrector stopped converging, the orbit shrank onto a Hopf point, or the period grew without bound (a homoclinic orbit is near). If it stopped because of a step-size or point limit, adjust the corresponding setting and run it again.

16 Saving and exporting results

Export SVG Saves the current view as a vector figure with light-theme colours, ready for a paper or for editing in Inkscape or Illustrator. All the plots in this document were made this way.

Export CSV Available in the Time course, Bifurcation and Two-parameter views.

- The time-course table has every trajectory on screen, one row per stored point, with the trajectory number, its kind (trajectory or manifold), its direction (forward or backward), t and every variable.
- The bifurcation table has one row per computed point, with these columns: curve, kind (equilibrium, periodic orbit, sweep, boundary value problem or special), label (HB1, LPC2, ...), parameter name and value, stability, number of unstable directions, period, the state, and the maximum and minimum of each variable along an orbit.
- The two-parameter table has the two parameter values, the period where relevant, the state, and labelled rows for codimension-two points. Hopf curves also get columns for ℓ_1 and for criticality (subcritical, supercritical or neutral saddle).
- In the bifurcation table, special rows for Hopf points give ℓ_1, and the stability column says subcritical or supercritical.

Export .dat In the Bifurcation and Two-parameter views, saves the same continuation data as `.dat` files: numbers only, no header and space-separated columns, the form XPPAUT and gnuplot read. The bifurcation diagram gives one file each for equilibria, periodic orbits, sweeps and boundary value branches. The boundary value file (`-bvp`) has the columns curve, parameter number, value, the initial values, and then the maximum, minimum and L^2 norm of each variable. The status line lists the columns, and text fields become numeric codes.

Copy CSV, Save .dat In the trajectory list, copy the current trajectory (time, variables and auxiliaries) to the clipboard, or save it as a `.dat` file.

Copy text Copies the `.ode` file, including any edits, so that it can be saved or used in XPPAUT.

Writing the current state into the file

After exploring a model you will often want to keep the parameter values and settings you have arrived at. **Write state**, in the Equations tab, writes them into the `.ode` text, much as XPPAUT does when it saves a file:

- parameter values and initial conditions are changed where they are defined, in the `par`, `init` and `x(0)=` lines, as are slider ranges in `#@range` and `#@irange` lines and history formulas of delay equations;
- the integration settings (method, total time, start time, step, transient, tolerance, bound, random seed), the plot axes and window, and the continuation settings are collected in `@` lines just before `done`.

For example, after setting $i = 150$ and a transient of 100 in the Morris–Lecar example, the end of the file reads

```
param i=150,c=20
...
# Settings written by XPP Studio, 2026-10-20 10:30
@ meth=rungekutta, total=400, dt=0.25, trans=100, bound=10000
@ xp=v, yp=w, zp=ica, xlo=-75, xhi=45, ylo=-0.1, yhi=0.55
@ ds=2, nmax=3000, parmin=0, parmax=300
done
```

Pressing **Write state** again replaces this block rather than adding another, and **Apply** on the result restores exactly the same state. The text is not reloaded, so the trajectories on screen are kept. Use **Copy text**, or your browser's own storage of the editor, to keep the file. Parameters defined through array lines are left as written.

17 Differences from XPPAUT

XPP Studio aims to read the same files and to feel familiar, but it is a reimplementation:

- **Continuation.** It is built in and written from scratch: pseudo-arclength continuation of equilibria, and multiple shooting (rather than AUTO's orthogonal collocation) for periodic orbits. Results agree with AUTO to plotting accuracy on the examples, but very long or very unstable orbits may need more shooting intervals (Ntst).
- **Delay equations** are integrated with a fixed step only, and periodic orbits of delay equations cannot be continued. XPPAUT's other delay options, such as `delshft` and `ndelay`, are not supported. The `@ delay` option is accepted but not needed, since all of the history is kept.
- **Global flags** can set state and Markov variables and `arret`. The XPP output flag `out_put` is accepted and ignored.
- **Boundary value problems** (`b` or `bndry` lines) are solved by shooting only, for ordinary differential equations, with one condition for each variable (Section 14). Their solutions can be continued in one parameter with pseudo-arclength steps, which finds folds but not branch points and gives no stability; two-parameter continuation of folds is not available.
- **Not supported:** `special` and `export` declarations, and `set` blocks.
- **Two-parameter Hopf curves** are limited to 12 variables, because the bialternate product grows quadratically with dimension.
- **Volterra models** cannot be continued, since their state is infinite-dimensional.
- **Browser files.** Files are opened and saved through the browser. Table data files are kept in the browser's local storage until you press **Forget table files**.

18 Credits and licence

XPP Studio is inspired by, and reads the model files of, XPPAUT by G. Bard Ermentrout (University of Pittsburgh), described in [1]. Its continuation methods follow the ideas of AUTO [2] by Eusebius Doedel and colleagues, and of MatCont [3]. This version (October 2026) was created by Peter Ashwin and Bard Ermentrout in collaboration with Claude Opus 5.5, an AI model made by Anthropic.

XPP Studio is free software, released under the GNU General Public License, version 3 or later. The licence text is distributed with the program (`LICENSE.txt`) and is available at `https://www.gnu.org/licenses/gpl-3.0.html`.

References

[1] G. B. Ermentrout, *Simulating, Analyzing, and Animating Dynamical Systems: A Guide to XPPAUT for Researchers and Students*, SIAM, Philadelphia, 2002.

[2] E. J. Doedel, A. R. Champneys, F. Dercole, T. F. Fairgrieve, Yu. A. Kuznetsov, B. Oldeman, R. C. Paffenroth, B. Sandstede, X. J. Wang and C. Zhang, *AUTO-07P: Continuation and bifurcation software for ordinary differential equations*, Concordia University, 2007.

[3] A. Dhooge, W. Govaerts and Yu. A. Kuznetsov, MATCONT: a MATLAB package for numerical bifurcation analysis of ODEs, *ACM Trans. Math. Software* **29** (2003) 141–164.

[4] W. J. F. Govaerts, *Numerical Methods for Bifurcations of Dynamical Equilibria*, SIAM, Philadelphia, 2000.

[5] Yu. A. Kuznetsov, *Elements of Applied Bifurcation Theory*, 3rd edn, Springer, New York, 2004.

[6] H. B. Keller, Numerical solution of bifurcation and nonlinear eigenvalue problems, in *Applications of Bifurcation Theory*, ed. P. H. Rabinowitz, Academic Press, 1977, pp. 359–384.

[7] L. F. Shampine and M. W. Reichelt, The MATLAB ODE suite, *SIAM J. Sci. Comput.* **18** (1997) 1–22.

[8] P. Linz, *Analytical and Numerical Methods for Volterra Equations*, SIAM, Philadelphia, 1985.

[9] C. Morris and H. Lecar, Voltage oscillations in the barnacle giant muscle fiber, *Biophys. J.* **35** (1981) 193–213.

[10] J. Rinzel and G. B. Ermentrout, Analysis of neural excitability and oscillations, in *Methods in Neuronal Modeling*, ed. C. Koch and I. Segev, MIT Press, 1989, pp. 135–169.

[11] D. Breda, S. Maset and R. Vermiglio, Pseudospectral differencing methods for characteristic roots of delay differential equations, *SIAM J. Sci. Comput.* **27** (2005) 482–495.

[12] G. E. Hutchinson, Circular causal systems in ecology, *Ann. N. Y. Acad. Sci.* **50** (1948) 221–246.

[13] M. C. Mackey and L. Glass, Oscillation and chaos in physiological control systems, *Science* **197** (1977) 287–289.

[14] O. E. Rössler, An equation for continuous chaos, *Phys. Lett. A* **57** (1976) 397–398.